PROJECT COMPASS

Technical Architecture Document (TAD)

Version: 0.1
Status: Proposed architecture
Based on: PRD v1.0 and Technical Discovery
Architecture style: Modular Monolith + AI Processing Service
Deployment approach: Cloud-managed infrastructure

---

1. Architecture Objectives

The architecture must support:

1. Chrome and Firefox browser extensions.
2. Android and iOS applications.
3. Offline-first information capture.
4. Reliable synchronization between devices.
5. Multi-provider AI integration.
6. Flexible personal organization through tags.
7. Future external integrations.
8. Subscription and feature entitlement management.
9. Secure storage of personal information.
10. Future integration with external AI assistants and platforms.

2. Technology Stack

Layer	Technology	Purpose
Mobile	React Native + Expo	Android and iOS
Browser	WXT + React	Chrome and Firefox
Language	TypeScript	Shared application logic
Backend	Node.js + NestJS	Core API
AI service	Python + FastAPI	Intelligence processing

Database	PostgreSQL	Primary persistence
Vector search	pgvector	Future semantic retrieval
Mobile storage	SQLite	Local persistence
Extension storage	IndexedDB	Offline persistence
API	REST / JSON	Communication
Authentication	Supabase Auth	Managed identity
Database hosting	Supabase PostgreSQL	Managed infrastructure
Object storage	Supabase Storage	Future attachments
Testing	Jest / Vitest / Playwright	Automated testing
Mobile testing	Maestro	Mobile E2E
API testing	Postman / Pytest	API and AI validation
CI/CD	GitHub Actions	Automated pipelines

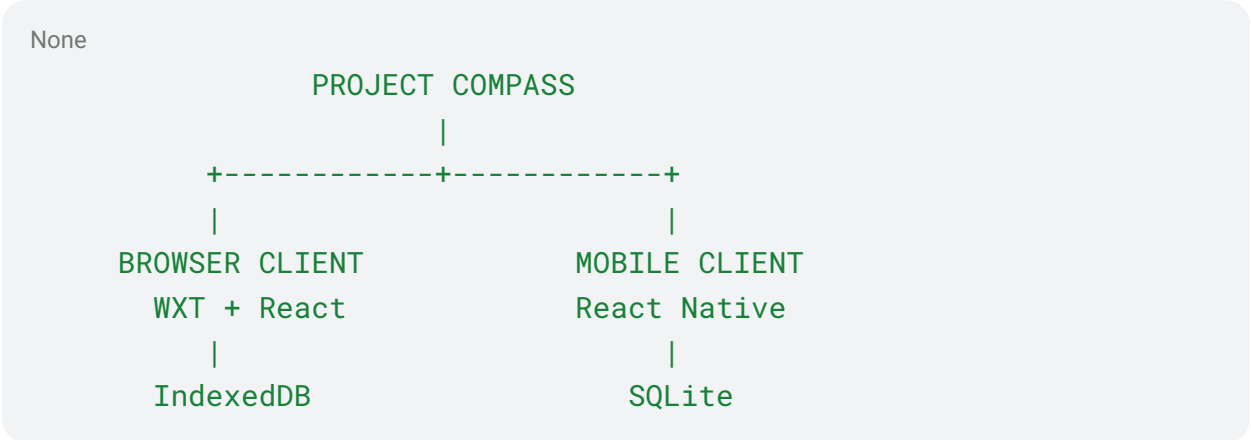
Architecture decision

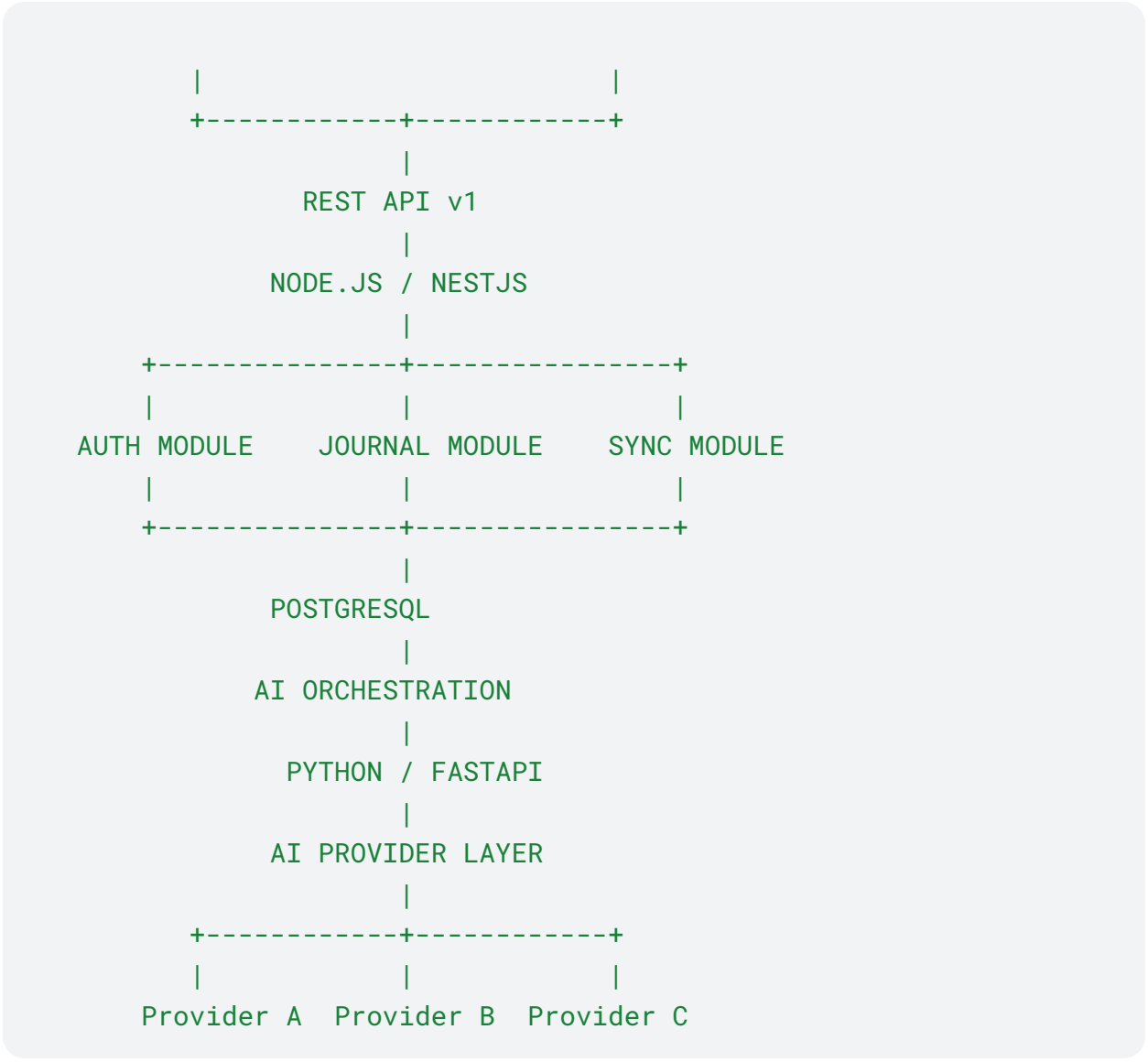
Use managed infrastructure for authentication and database hosting, but keep application business logic in our own backend.

This reduces initial cloud complexity without giving up control of the product.

3. High-Level Architecture

None





The Node.js backend owns business rules and data persistence.

Python handles intelligence-related operations.

AI providers are replaceable integrations, not permanent architectural dependencies.

4. Application Components

4.1 Browser Extension

Framework: WXT + React + TypeScript.

Responsibilities:

- Quick capture.
- Selected-text extraction.
- URL and page-title extraction.
- Task creation.
- Reference creation.
- Local offline queue.
- Synchronization.

Browser permissions should follow least-privilege principles.

The extension should request access to the active tab when needed instead of unrestricted browsing access.

4.2 Mobile Application

Framework: React Native + Expo.

Responsibilities:

- User authentication.
- Inbox management.
- Daily log.
- Task management.
- Tag management.
- Search.
- Reflection.
- Local persistence.
- Synchronization.

Mobile navigation should prioritize capture, inbox, daily log, and search.

4.3 Core Backend

Framework: NestJS.

Initial modules:

- AuthModule
- UsersModule
- DevicesModule
- EntriesModule
- TasksModule
- TagsModule
- SearchModule
- SyncModule

- AIOrchestrationModule
- SubscriptionModule
- IntegrationsModule
- AuditModule

These are logical modules within one deployable backend, not separate microservices.

4.4 AI Processing Service

Framework: Python + FastAPI.

Responsibilities:

- Natural-language interpretation.
- Classification.
- Date extraction.
- Priority recommendations.
- Duplicate detection.
- Personalized recommendations.

The service must expose a provider-independent interface.

AI responses must use validated structured output.

The AI service cannot directly modify user records. It returns proposed changes to the Node.js backend, which applies authorization, validation, and persistence rules.

4.5 Shared Data Layer

PostgreSQL is the authoritative cloud data store.

Local databases provide immediate access and offline functionality.

The architecture must preserve:

- Unique record identifiers.
- Version information.
- Synchronization status.
- Original user input.
- Data ownership.
- Creation and modification timestamps.

5. Offline and Synchronization Strategy

Approach: Local-first with cloud synchronization.

Local persistence

Mobile:

- SQLite.

Browser:

- IndexedDB.

Synchronization flow

1. User creates or modifies an entry.
2. Client saves the change locally.
3. Client creates a synchronization operation.
4. When connected, the client submits pending operations.
5. Backend validates ownership and version.
6. Backend persists the change.
7. Backend returns the canonical record and synchronization cursor.
8. Client updates local state.

Conflict resolution

For the MVP:

- Use optimistic concurrency with record versions.
- Never silently overwrite conflicting user edits.
- Use deterministic resolution for non-conflicting changes.
- Request user intervention when concurrent edits cannot be safely reconciled.
- Make synchronization operations idempotent.

Synchronization target

The initial product does not require real-time synchronization.

It should synchronize:

- When the application opens.
- When connectivity returns.
- After meaningful user changes, when connected.
- When the user manually requests synchronization.

6. AI Architecture

6.1 Provider abstraction

Create an internal interface:

`AIProvider`

Required operations:

- interpretEntry()
- classifyEntry()
- detectDates()
- suggestPriority()
- detectDuplicates()
- generateRecommendations()

Each provider adapter must normalize its responses into a common internal format.

6.2 AI processing pipeline

```
None
USER INPUT
    |
    v
INPUT VALIDATION
    |
    v
CONTEXT RETRIEVAL
    |
    v
AI PROVIDER
    |
    v
STRUCTURED RESPONSE
    |
    v
SCHEMA VALIDATION
    |
    v
CONFIDENCE EVALUATION
    |
    +------ Low confidence ------> USER REVIEW
```

```
    |
    +------ High confidence -----> APPLY ACCORDING TO USER MODE
    |
    v
PERSISTENCE
```

6.3 Autonomy rules

Default behavior: Semi-autonomous.

High-confidence suggestions can be automatically applied according to the user's settings.

Low-confidence or ambiguous suggestions must be presented for review.

Users can choose preview mode or automatic mode.

The original input must always remain available.

6.4 AI safety and reliability

- Validate all model responses.
- Do not execute arbitrary model-generated instructions.
- Do not allow AI to bypass user permissions.
- Prevent automatic deletion of records.
- Do not permit unapproved external actions.
- Log processing outcomes without unnecessarily storing sensitive prompts.
- Provide a fallback when AI services are unavailable.

7. Security Architecture

Authentication

Use Supabase Auth for managed identity.

The NestJS backend validates authentication tokens and independently enforces authorization.

Authorization

All user-owned records must be scoped by user ID.

A user must never access another user's journal data through an API request.

Data protection

- HTTPS/TLS for network communication.
- Encryption at rest through managed infrastructure.
- Secure token storage.
- Environment-based secrets.
- No credentials in frontend source code.
- Restricted database permissions.
- Rate limiting.
- Audit logging for sensitive operations.

Browser security

- Minimal extension permissions.
- Content Security Policy.
- Validate messages between extension components.
- Avoid unrestricted page-content collection.

Privacy

AI processing is optional.

Users can disable AI and control which information can be sent for processing.

External AI providers must be documented in the privacy policy.

8. Subscription and Feature Architecture

The application must separate product entitlements from subscription payment providers.

Proposed plans:

Basic

- Manual capture.
- Daily journal.
- Tasks.
- Tags.
- Basic synchronization.
- Export.

AI

- Natural-language capture.
- Automatic classification.

- Date detection.
- Priority suggestions.
- Duplicate detection.

Advanced

- Personalized recommendations.
- Advanced integrations.
- Increased AI usage.
- Expanded assistant capabilities.

Initial commercial hypothesis:

- Free trial.
- US$8.99/month.
- US$14.99/month.

These are proposed pricing points, not validated prices.

The backend must enforce entitlements and AI usage limits.

9. External Integrations

Future integration candidates:

- Google Calendar.
- Apple Calendar.
- Gmail.
- Outlook.
- Slack.
- Browser services.
- External AI assistants.

Integrations must use authorized OAuth connections where supported.

The architecture should support an external integration adapter.

Future possibility: an MCP-compatible interface exposing selected Project Compass capabilities to external AI assistants.

This should be evaluated as a separate product extension, not included in the first MVP.

10. Infrastructure and Deployment

Recommended initial environment:

- Supabase: authentication and managed PostgreSQL.
- Node.js backend: containerized deployment.
- Python AI service: containerized deployment.
- GitHub Actions: automated testing and deployment.
- Expo EAS: mobile builds.

Development, staging, and production must have separate configurations and secrets.

The initial budget target is US$10–30/month where feasible.

Free-tier availability and provider pricing must be checked before deployment. AI inference costs should be monitored separately.

11. Monitoring and Observability

Track:

- API availability.
- API latency.
- Authentication errors.
- Synchronization failures.
- AI processing latency.
- AI provider errors.
- AI usage and estimated costs.
- Database performance.

Do not log private journal content by default.

12. Architectural Evolution

MVP:

- Modular monolith.
- Managed PostgreSQL.
- Single AI processing service.
- Local-first capture.

Growth:

- Background job processing.
- Advanced search.
- Calendar and email integrations.
- Improved AI personalization.

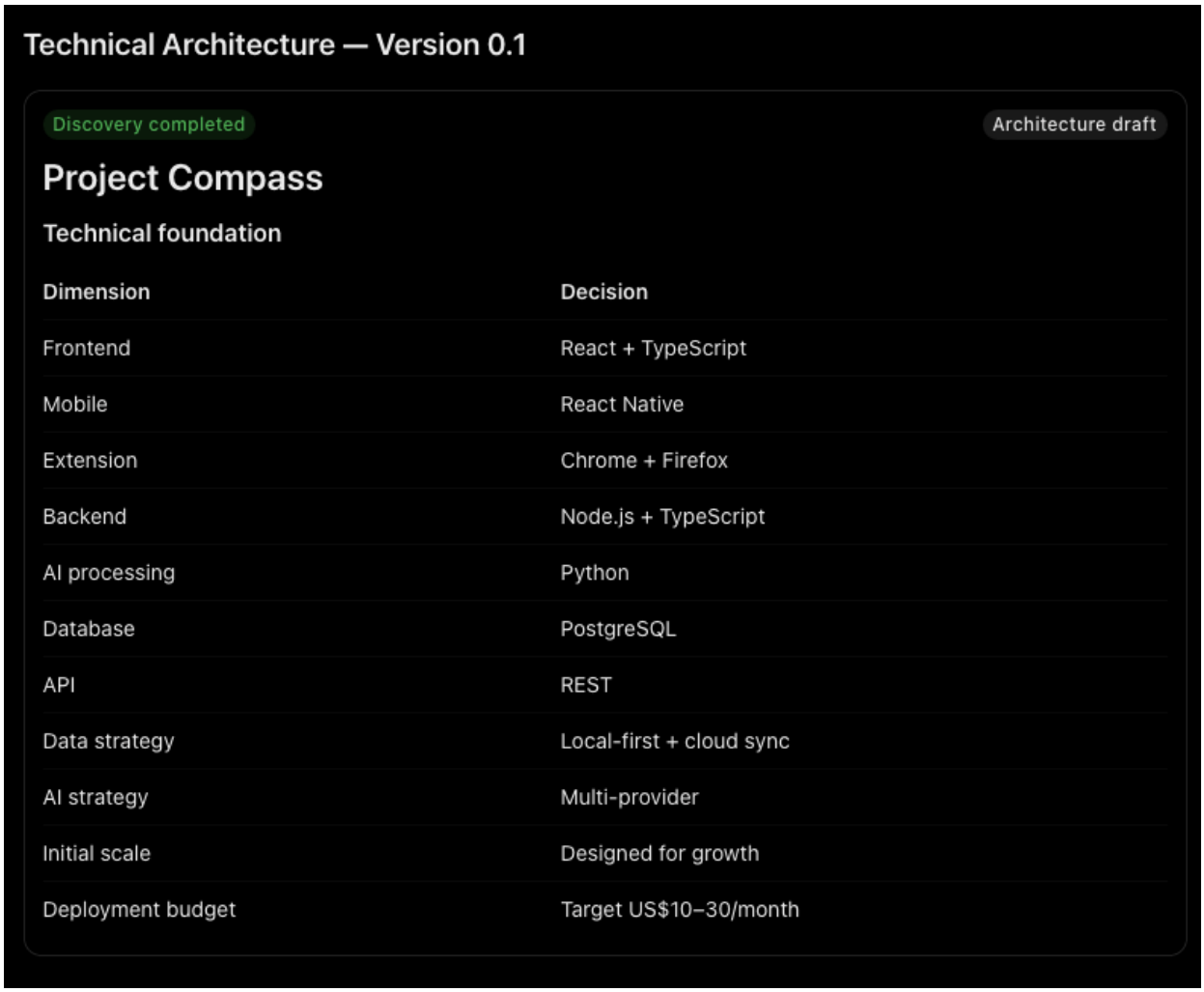
Scale:

- Dedicated AI workers.
- Independent synchronization infrastructure if justified.
- External assistant integrations.
- Advanced privacy and encryption capabilities.

Architecture principle:

Keep the core product simple, modular, secure, provider-independent, and extensible.

—---



My recommended technology stack



React + TypeScript

Shared frontend engineering foundation. We'll use React for the extension and React Native for mobile, with reusable TypeScript packages for common business logic.



React Native + Expo

One mobile codebase targeting Android and iOS. This is a good balance between development speed and native capabilities.



WXT + React

Browser extension framework supporting Chromium-based browsers and Firefox, with shared extension code.



Node.js + NestJS

Structured backend architecture with modules, dependency injection, authentication, validation, and REST endpoints.



Python + FastAPI

Dedicated AI processing interface. It will support provider adapters, semantic interpretation, and evaluation of AI outputs.



PostgreSQL + pgvector

Relational database for reliable records and relationships, with vector search available for future semantic retrieval.
